- *Binding model*—This would take the `{query}` route parameter from the URL and any values posted in the request body (maybe a sort order, or the number of items to show), and bind them to a C# class, which typically acts as a throwaway data transport class. This would be set as a property on the `PageModel` when the page handler is invoked.
- *Application model*—This is the services and classes that perform the logic. When invoked by the page handler, this model would load all the clothes that match the query, applying the necessary sorting and filters, and return the results to the controller.
- *Page model*—The values provided by the application model would be set as properties on the Razor Page's `PageModel`, along with other metadata, such as the total number of items available or whether the user can currently check out. The Razor view would use this data to render the Razor view to HTML.

The important point about all these models is that their responsibilities are well defined and distinct. Keeping them separate and avoiding reuse helps ensure that your application stays agile and easy to update.

The obvious exception to this separation is the `PageModel`, as it is where the binding models and page handlers are defined, and it also holds the data required for rendering the view. Some people may consider the apparent lack of separation to be sacrilege, but it's not generally a problem. The lines of demarcation are pretty apparent. So long as you don't try to, for example, invoke a page handler from inside a Razor view, you shouldn't run into any problems!

Now that you've been properly introduced to the various models in ASP.NET Core, it's time to focus on how to use them. This chapter looks at the binding